
UT04

POO

en Python

Programación Orientada a Objetos (POO) en Python

Introducción

Programación orientada a objetos y programación estructurada

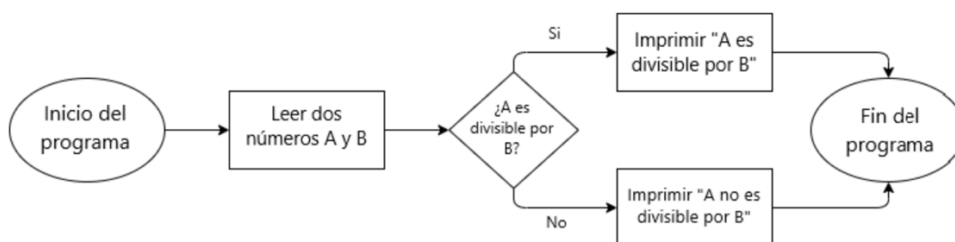
Como la mayoría de las actividades que hacemos a diario, la programación también tiene diferentes formas de realizarse. Estos modos se llaman **paradigmas de programación** y entre ellos están la **programación orientada a objetos** (POO) y la **programación estructurada**. Cuando comenzamos a usar lenguajes como Java, C#, Python y otros que posibilitan el paradigma orientado a objetos, es común cometer errores y aplicar la programación estructurada pensando que estamos usando recursos de la orientación a objetos.

En la **programación estructurada**, un programa consta de tres tipos básicos de estructuras:

- Secuencias: son los comandos a ejecutar
- Condiciones: secuencias que solo deben ejecutarse si se cumple una condición (ejemplos: *if-else*, *match(switch)* y comandos similares)
- Repeticiones: secuencias que deben realizarse repetidamente hasta que se cumpla una condición (*for*, *while*, *do-while*, etc.)

Estas estructuras se utilizan para procesar la entrada del programa, cambiando los datos hasta que se genera la salida esperada. Hasta ahora, nada que la programación orientada a objetos no haga también, ¿verdad?

La principal diferencia es que, en la programación estructurada, un programa generalmente se escribe en **una sola rutina** (o función) y, por supuesto, puede dividirse en subrutinas. Pero el flujo del programa sigue siendo el mismo, como si se pudiese copiar y pegar el código de las subrutinas directamente en las rutinas que las llaman, de tal forma que, al final, solo existiese una gran rutina que ejecute todo el programa.



Además, el acceso a las variables no tiene muchas restricciones en la programación estructurada. En lenguajes fuertemente basados en este paradigma, restringir el acceso a una variable se limita a decir si es visible o no dentro de una función (o módulo, como en el uso de la palabra clave *static*, en lenguaje C), pero no es posible decir de forma nativa que solo se puede acceder a una variable mediante unas pocas rutinas del programa. El esquema para situaciones como estas implica prácticas de programación perjudiciales para el desarrollo del sistema, como el **uso excesivo de variables globales**. Vale la pena

recordar que las variables globales se usan típicamente para mantener estados en el programa, marcando en qué parte de la ejecución se encuentran.

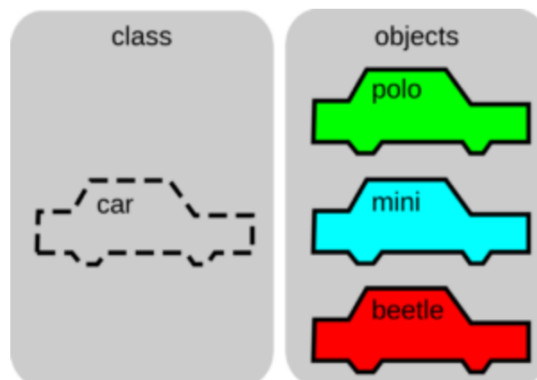
La **programación orientada a objetos** surgió como una alternativa a estas características de la programación estructurada. El propósito de su creación fue también acercar el manejo de las estructuras de un programa al manejo de las cosas en el mundo real, de ahí el nombre "objeto" como algo genérico, que puede representar cualquier cosa tangible.

Este nuevo paradigma se basa principalmente en dos conceptos clave: **clases** y **objetos**. Todos los demás conceptos, igualmente importantes, se basan en estos dos.

¿Qué son clases y objetos?

Imagina que recientemente compraste un coche y decidiste **modelar ese coche utilizando programación orientada a objetos**. Tu coche tiene las características que estabas buscando: motor 2.0 híbrido, azul oscuro, cuatro puertas, cambio automático, etc. También tiene comportamientos que probablemente fueron el motivo de tu compra, como acelerar, reducir la velocidad, encender los faros, tocar la bocina y tocar música. Podemos decir que el coche nuevo es un *objeto*, donde sus características son sus *atributos* (datos vinculados al objeto) y sus comportamientos son acciones o *métodos*.

Tu coche es un objeto tuyo, pero en la tienda donde lo compraste había otros tantos, muy similares, con cuatro ruedas, volante, cambio, espejos retrovisores, faros, entre otras partes. Ten en cuenta que, aunque tu coche es único (por ejemplo, tiene un registro único en el Departamento de Tránsito), puede haber otros con exactamente los mismos atributos, o similares, o incluso totalmente diferentes, pero que aún se consideran *autos*. Entonces podemos decir que tu objeto se puede clasificar (es decir, tu *objeto pertenece a una clase*) como un coche, y que tu coche no es más que una instancia de esa clase llamada "coche".



Así, abstrayendo un poco la analogía, **una clase es un conjunto de características y comportamientos que definen el conjunto de objetos pertenecientes a esta clase**.

Tenga en cuenta que la clase en sí es un concepto abstracto, como un molde, que se vuelve concreto y palpable a través de la creación de un objeto. Llamamos a esta creación de **instanciación de clase**, como si estuviéramos usando este molde (clase) para crear un objeto.

```
1 class Auto:
2     def __init__(self, modelo) -> None:
3         self.modelo = modelo
4         self.velocidad = 0
5
6     def acelerar(self):
7         pass
8
9     def frenar(self):
10        pass
11
12    def encenderFaro(self):
13        pass
```

Resumiendo:

Un **objeto** es “algo” que se quiere representar, pero no es suficiente con una variable de los tipos más básicos (int, bool, string, etc.). Esta representación estará basada en los atributos que serán las características propias que lo definen y los métodos que serán las acciones que se podrán realizar con los objetos.

Los atributos en general serán variables de los tipos ya conocidos de Python, pero también pueden ser de otro tipo de objetos. Estas variables pueden tener distintos valores dependiendo para distintos objetos que se trate.

Normalmente estos métodos serán funciones de Python que pueden recibir parámetros o retornar valores.

Las **clases** son una estructura (molde) que permite crear (instanciar) objetos que tienen los mismos atributos y métodos, es decir, que podemos tener objetos iguales, pero con diferentes estados. Es decir que los objetos son iguales en estructura, pero con diferentes valores.

En Python una clase se define mediante la palabra clave “**class**” de la siguiente forma:

```
1 class Gato:
2     pass
```

Los objetos se crean instanciando la clase de la siguiente forma:

```
4 gatopardo = Gato()  
5  
6 # ver el tipo de la variable gatopardo  
7  
8 print("\n", type(gatopardo))
```

Veamos un ejemplo un poco más elaborado.

Se quiere representar información de los gatos para lo que se quiere crear una clase de nombre Gato en la que se quiere guardar el nombre, la edad y los alimentos favoritos que come, el sexo y la fecha de su última vacunación. Además, se quiere tener un método que indique si un gato es adulto y saber si un alimento es favorito para un gato.

Analizando la información podemos destacar que se deben crear tres atributos: nombre, edad y una lista de los alimentos favoritos del gato. Por otro lado, se debe tener un método que indique si un gato es adulto, y otro que reciba un parámetro que representa un alimento y nos indique si éste es uno de los favoritos del gato.

Además, se debe tener un método constructor que es el que permita crear los objetos, este método deberá recibir tres parámetros: el nombre, la edad y el sexo.

El código en Python que crea la clase es el que se muestra a continuación:

```
3 class Gato:  
4     def __init__(self, nombre, edad, sexo) -> None:  
5         self.nombre = nombre  
6         self.edad = edad  
7         self.sexo = sexo  
8         self.alimentos_favoritos = []  
9         self.vacunado = (False, "00/00/00")  
10  
11     def verEtapaDeVida(self):  
12         if self.edad > 1:  
13             print(self.nombre + " es adulto")  
14         else:  
15             print(self.nombre + " es cachorro")  
16  
17     def esAlimentoFavorito(self, alimento):  
18         return alimento in self.alimentos_favoritos
```

Como se puede apreciar el método constructor en Python se representa mediante el nombre

`__init__(self, parámetros_necesarios)`

En la definición de la clase Gato, se usan tres atributos: nombre, edad y sexo. Los métodos son los que se han creado mediante las dos definiciones que aparecen a continuación.

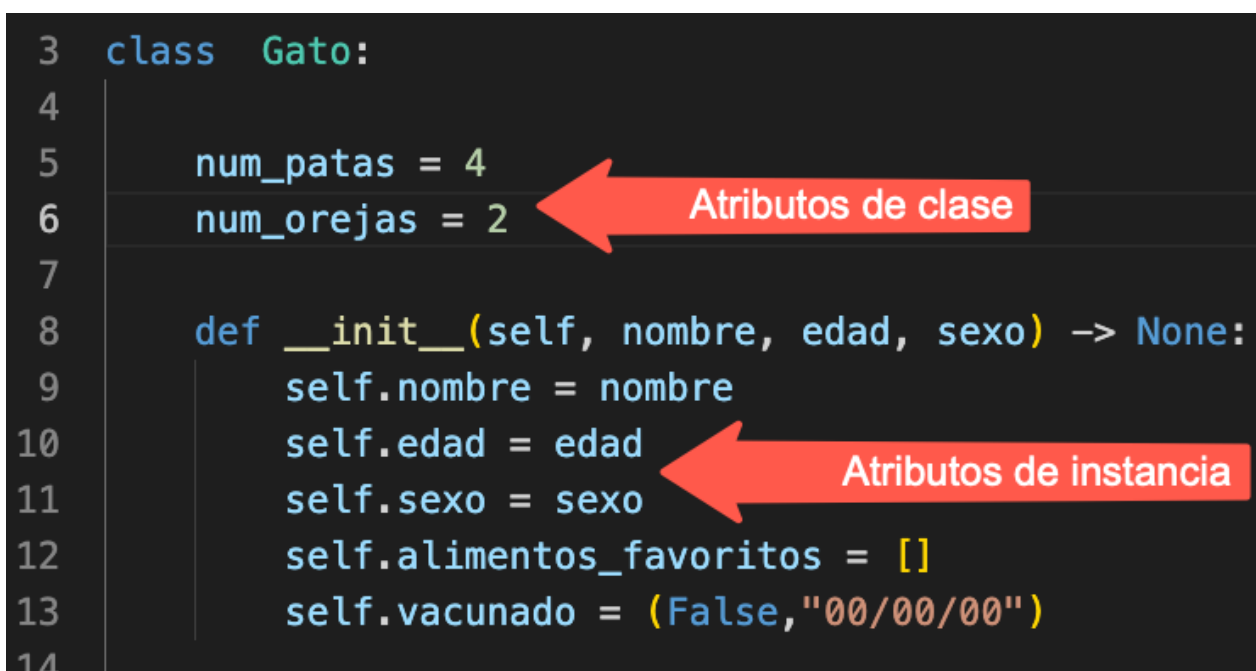
Destacar, que el nombre de las clases debe empezar con letra mayúscula, y los nombres de los métodos deben utilizar la notación **CamelCase**, es decir, debe comenzar en minúscula y cada palabra que lo conforma debe comenzar en mayúsculas, por ejemplo: *esAlimentoFavorito*, *verEtapaDeVida*.

Tipos de atributos

Los atributos pueden ser de dos tipos: de clase y de instancia.

Los atributos que se han creado en el ejemplo anterior son los **atributos de instancia** y **NO** son compartidos entre los objetos de la misma clase, de hecho, estos atributos se pueden crear dinámicamente en Python. Además de los atributos anteriores, también se pueden crear otros atributos que, si pueden ser compartidos por todos los objetos de la clase, son los **atributos de clase** y que pueden tener información común para todos los objetos de la clase.

Los atributos de clase se crean en la definición de la clase añadiendo los atributos en el primer nivel del bloque lógico, tal y como se muestra en el siguiente ejemplo:



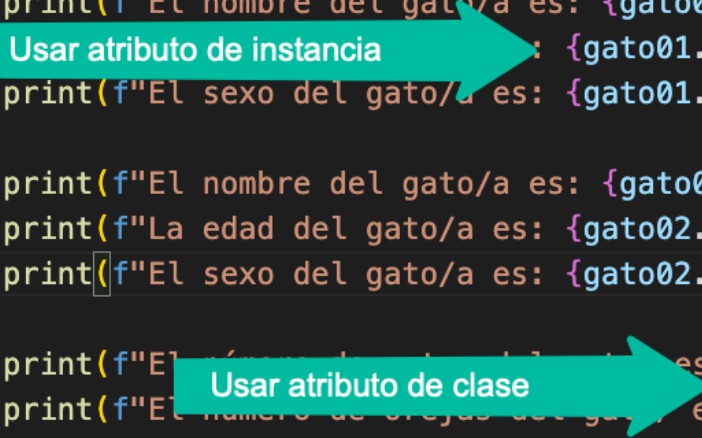
```
3 class Gato:
4
5     num_patas = 4
6     num_orejas = 2
7
8     def __init__(self, nombre, edad, sexo) -> None:
9         self.nombre = nombre
10        self.edad = edad
11        self.sexo = sexo
12        self.alimentos_favoritos = []
13        self.vacunado = (False, "00/00/00")
14
```

The image shows a code editor with Python code for a class named `Gato`. Lines 5 and 6 define class attributes `num_patas = 4` and `num_orejas = 2`. Lines 8-13 define the `__init__` method, which assigns instance attributes `self.nombre`, `self.edad`, `self.sexo`, `self.alimentos_favoritos`, and `self.vacunado`. A red arrow points from the text "Atributos de clase" to lines 5 and 6. Another red arrow points from the text "Atributos de instancia" to line 10.

Los atributos `num_patas` y `num_orejas` que están definidos antes del `__init__()` son los atributos de clase.

La forma de usar los atributos de instancia y los de clase se muestra en la siguiente figura:

```
53 # programa principal
54
55 def main():
56     gato01 = Gato("zarpas", 1, 'M')
57     gato02 = Gato("frida", 2, 'H')
58
59     print(f"El nombre del gato/a es: {gato01.nombre}")
60     print(f"La edad del gato/a es: {gato01.edad}")
61     print(f"El sexo del gato/a es: {gato01.sexo}")
62
63     print(f"El nombre del gato/a es: {gato02.nombre}")
64     print(f"La edad del gato/a es: {gato02.edad}")
65     print(f"El sexo del gato/a es: {gato02.sexo}")
66
67     print(f"El número de patas del gato/a es: {Gato.num_patas}")
68     print(f"El número de orejas del gato/a es: {Gato.num_orejas}")
69
```



La salida que muestra el programa es la siguiente:

```
El nombre del gato/a es: zarpas
La edad del gato/a es: 1
El sexo del gato/a es: M
El nombre del gato/a es: frida
La edad del gato/a es: 2
El sexo del gato/a es: H
El número de patas del gato/a es: 4
El número de orejas del gato/a es: 2
```

Los atributos de instancia pueden ser modificados por las instancias y dependiendo de si el tipo del atributo es mutable o no, pueden cambiar el valor.

```
53 # programa principal
54
55 def main():
56     gato01 = Gato("zarpas", 1, 'M')
57     gato02 = Gato("frida", 2, 'H')
58
59     gato01.edad = 2
60     gato02.edad = 3
61
62     print(f"El nombre del gato/a es: {gato01.nombre}")
63     print(f"La edad del gato/a es: {gato01.edad}")
64     print(f"El sexo del gato/a es: {gato01.sexo}")
65
66     print(f"El nombre del gato/a es: {gato02.nombre}")
67     print(f"La edad del gato/a es: {gato02.edad}")
68     print(f"El sexo del gato/a es: {gato02.sexo}")
```

Las líneas 59 y 60 indican como se puede cambiar el valor de un atributo de instancia. En la siguiente imagen que muestra la nueva salida de la ejecución del programa podemos ver que el cambio de los atributos de instancia sólo afecta a la instancia u objeto al que se le aplica el cambio.

```
El nombre del gato/a es: zarpas
La edad del gato/a es: 2
El sexo del gato/a es: M
El nombre del gato/a es: frida
La edad del gato/a es: 3
El sexo del gato/a es: H
```

En cambio, con los atributos de clase, si la modificación es realizada desde la clase, un cambio en ellos se reflejará en todos los objetos creados. Si el cambio se hace a través de una instancia, entonces el atributo de clase cambia el valor sólo para la instancia que lo modifica.

Ejemplo de un código que ejemplifica el cambio realizado desde la clase.


```
55 def main():
56     gato01 = Gato("zarpas", 1, 'M')
57     gato02 = Gato("frida", 2, 'H')
58
59     Gato.num_patas = 3
60
61     print(f"El número de patas del gato/a es: {gato01.num_patas}")
62     print(f"El número de patas del gato/a es: {gato02.num_patas}")
63
```

Que al ejecutar muestra la siguiente salida:

```
El número de patas del gato/a es: 3
El número de patas del gato/a es: 3
```

Se puede apreciar que el cambio hecho desde la clase afecta a todos los objetos que se hayan instanciados de ella.

Ejemplo de un código que ejemplifica el cambio realizado desde un objeto.

```
55 def main():
56     gato01 = Gato("zarpas", 1, 'M')
57     gato02 = Gato("frida", 2, 'H')
58
59     gato01.num_patas = 3
60
61     print(f"El número de patas del gato/a es: {gato01.num_patas}")
62     print(f"El número de patas del gato/a es: {gato02.num_patas}")
```

Cuyo resultado es el siguiente:

```
El número de patas del gato/a es: 3
El número de patas del gato/a es: 4
```

Se puede apreciar que ahora que el cambio realizado a un atributo de clase desde una instancia no afecta al resto de las instancias.

Nombre y privacidad de en las clases

Para los atributos y nombres de métodos en Python

- `'_'` (1 guion bajo) como prefijo: cuando se utiliza un solo carácter `'_'` significa que ese atributo o método debe considerarse protegido para la clase y no se debería usarse fuera de la misma.
- `'__'` (2 guiones bajos) como prefijo: cuando se utiliza dos caracteres `'__'` significa que el atributo o método es privado y se requiere encarecidamente que no se use fuera del ámbito de la clase. Python implementa lo que se denomina **'name mangling'** que consiste en el cambio de nombre automático a estos atributos realizado por Python.

Destacar que en Python no existen los conceptos de privacidad que hay en otros lenguajes como Java, en el que se pueden definir propiedades como "privado" y "protegido". Veamos un ejemplo de la manipulación de los nombres.

```

1  class Foo:
2
3      _atributo_clase_protegido = 0
4      __atributo_clase_privado = 0
5
6      def __init__(self, x) -> None:
7          self.x = x
8          self._x = 2 * x
9          self.__x = 3 * x
10
11     def obtener_x(self):
12         return self.x
13
14     def obtener_x_protegida(self):
15         return self._x
16
17     def obtener_x_privada(self):
18         return self.__x
19
20     # principal
21
22     instancia = Foo(2)
23
24     print('Atributo normal ', instancia.obtener_x())
25     print('Atributo protegido ', instancia.obtener_x_protegida())
26     print('Atributo privado ', instancia.obtener_x_privada())
27     print("*****")
28     print('Atributo normal ', instancia.x)
29     print('Atributo protegido ', instancia._x)
30     print('Atributo privado ', instancia.__x)
31

```

```

Atributo normal 2
Atributo protegido 4
Atributo privado 6
*****
Atributo normal 2
Atributo protegido 4
Traceback (most recent call last):
  File "/Users/rgh.informatica/Documents/ACADEMICO/CURSO_21-22/DAM/00-CARPETA-CORRECCION/privacidad.py", line 30, in <module>
    print('Atributo privado ', instancia.__x)
AttributeError: 'Foo' object has no attribute '__x'. Did you mean: '_x'?

```

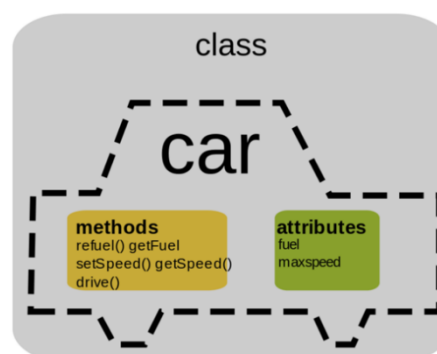
Fijarse que la línea 30 produce un error ya que no se reconoce el atributo '`__x`'; se recomienda no usar los atributos con el carácter '`_`' fuera del alcance de la clase. Para ello es mejor crear métodos que nos permitan acceder y modificar sus valores.

Encapsulamiento, herencia y polimorfismo: las principales características de la POO

Las dos bases de la POO son los conceptos de **clase y objeto**. De estos conceptos derivan algunos otros conceptos extremadamente importantes al paradigma, que no solo lo definen, sino que son las soluciones a algunos problemas de la programación estructurada. Los conceptos en cuestión son el **encapsulamiento**, la **herencia**, las **interfaces** y el **polimorfismo**.

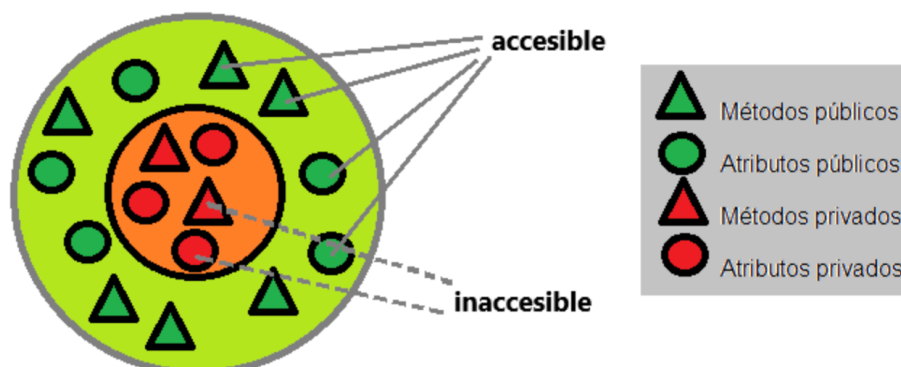
Encapsulamiento

Aun usando la analogía del coche, sabemos que tiene **atributos y métodos**, es decir, características y comportamientos. Los métodos del coche, como acelerar, pueden usar atributos y otros métodos del coche, como el tanque de gasolina y el mecanismo de inyección de combustible, respectivamente, ya que acelerar consume combustible.



Sin embargo, si algunos de estos atributos o métodos son fácilmente visibles y modificables, como el mecanismo de aceleración del coche, esto puede darle libertad para realizar cambios, lo que resulta en efectos secundarios imprevisibles. En esta analogía, una persona puede no estar satisfecha con la aceleración del coche y cambia la forma en que se produce, creando efectos secundarios que pueden hacer incluso con que el coche ni ande, por ejemplo.

En este caso, decimos que el método de aceleración de su coche no es visible desde el exterior del mismo coche. En la POO, un atributo o método que no es visible desde afuera del mismo objeto se llama "**privado**" y cuando está visible, se llama "**público**".



Pero entonces, ¿cómo sabemos cómo acelera nuestro coche? Es simple: no lo sabemos. Solo sabemos que para acelerar hay que pisar el acelerador y el objeto sabe cómo realizar esta acción sin exponer cómo lo hace. Decimos que la aceleración del coche está **encapsulada**, porque sabemos lo que hará cuando ejecutemos este método, pero no sabemos cómo y, de hecho, al programa **no le importa cómo lo hace el objeto, solo que lo haga**.

Lo mismo ocurre con los atributos. Por ejemplo: no sabemos cómo el coche sabe qué velocidad mostrar en el velocímetro o cómo calcula su velocidad, pero no necesitamos saber cómo se hace eso. Solo necesitamos saber que nos dará la velocidad adecuada. Se puede leer o cambiar un atributo encapsulado desde **getters y setters**.

Ese encapsulamiento de atributos y métodos evita la llamada *fuga de alcance*, donde un atributo o método es visible para alguien que no debería poder verlo, como otro objeto o clase. Esto evita confundir el uso de variables globales en el programa, facilitando identificar en qué estado estará cada variable en cada momento del programa, ya que la restricción de acceso nos permite identificar quién puede modificarla.

Getters (decorador @property)

En muchas ocasiones es necesario tener un control extra sobre los atributos y definir como se deben acceder, como se pueden actualizar y que hacer cuando se quieren eliminar. Para este propósito se utiliza los **getters** y los **setters** y los **deletters**.

Getter está implementado en python mediante un decorador (@property) de la función property el cual permite definir accesos, actualizaciones, eliminación y documentación. La sintaxis es la siguiente:

@property: sirve para definir el acceso al atributo y definir la cadena de caracteres que se usará como documentación. Si solamente se define la propiedad utilizando este decorador, se crea una versión de sólo lectura del mismo.

@<atributo>.setter: sirve para definir como actualizar el atributo. Si no se define este decorador, el atributo no puede ser modificado.

@<atributo>.deleter: sirve para definir como eliminar el atributo. Si no se define este decorador, el atributo no puede ser eliminado.

Veamos el siguiente ejemplo:

Se quiere definir la clase Círculo que tiene como atributo el radio

La codificación de la clase es la que se muestra:

```
1  # circle.py
2
3  class Circulo:
4      def __init__(self, valor_radio):
5          self._radio = valor_radio
6
7      @property
8      def _radio(self):
9          print("pasando por el getter")
10         return self.__radio
11
12     @_radio.setter
13     def _radio(self, nuevo_valor):
14         if nuevo_valor > 0:
15             print("Paso por el setter")
16             self.__radio = nuevo_valor
17         else:
18             print("El radio debe tener un valor > 0")
19
20     @_radio.deleter
21     def _radio(self):
22         print("Entrando en el deleter")
23         del self.__radio
```

Los métodos para acceder y modificar el valor del radio son los siguientes:

```
25     def obtenerRadio(self):
26         return self._radio
27
28     def actualizarRadio(self, nuevo_radio):
29         self._radio = nuevo_radio
```

Comentemos el código

Atributos de sólo lectura

Un atributo se considera que sólo es de lectura, cuando se le define el getter pero no el setter.

Atributo de solo escritura

Un atributo se considera que sólo es de escritura, cuando se le define el setter pero no el getter.

Métodos de clase

Los métodos de clase están orientados a operar a nivel de clase y no de instancia. Para poder definir un método de clase es necesario decorar el método con *@classmethod* (usar un decorador). El primer parámetro se llama, por convención, *cls*.

Veamos un ejemplo:

```

1 class Foo:
2     x = 10
3
4     def __init__(self,x ) -> None:
5         self.x = x
6
7     @classmethod
8     def get_x_class(cls):
9         return cls.x
10
11 # principal
12
13 f = Foo(-2)
14 print(f.x)
15 print("****")
16
17 print("---",Foo.get_x_class())
18
19 print(f.get_x_class())

```

```

rgh.informatica@macbpro-
-2
****
--- 10
10
rgh.informatica@macbpro-

```

En el ejemplo anterior, clase e instancia comparten el atributo. Cuando se inicializa la instancia *f*, se le asigna el valor -2 al atributo *x* de la instancia, pero como se aprecia cuando se pregunta por *get_x_class*, el valor devuelto es el de la clase y no el de la instancia.

Un caso particular del uso de métodos de clases se presenta cuando se quieren hacer instancias de una clase que se crean de forma diferente al constructor ordinario.

Veamos un ejemplo práctico.

Se pretende crear la clase *Animal* que modele animales con distinto nombre. Los atributos que se pretende guardar son: tipo, volumen y masa.

Se quiere instanciar objetos de la manera tradicional, pero que también se puedan inicializar instancias si se provee una cadena de caracteres separada por comas con los valores de los atributos necesarios. Adicionalmente, queremos tener métodos simples que construyan un gato o un perro. Para ello se sabe que un gato es de tipo 'Gato', tiene un volumen de 120 cm y una masa de 3,8 Kg y un perro es de tipo 'Perro', tiene un volumen de 500 cm y una masa de 25,4 Kg.

La clase que se desarrolla es la siguiente:

```

1  class Animal:
2      def __init__(self, tipo, volumen, masa) -> None:
3          self.tipo = tipo
4          self.volumen = float(volumen)
5          self.masa = float(masa)
6
7      @classmethod
8      def desde_str(cls, cadena):
9          tipo, volumen, masa = cadena.split(',')
10         return cls(tipo, float(volumen), float(masa))
11
12     @classmethod
13     def gato(cls):
14         return cls('Gato', 120.0, 3.8)
15
16     @classmethod
17     def perro(cls):
18         return cls('Perro', 500.0, 25.4)
19
20     def animalToString(self):
21         return self.tipo + ' ' + str(self.volumen) + ' ' + str(self.masa)
22

```

Un programa para usar la clase anterior:

```

25  cebra = Animal('Cebra', 15000, 150)
26  elefante = Animal.desde_str('Elefante,300000,2600')
27  gato = Animal.gato()
28  perro = Animal.perro()
29
30  print(cebra.animalToString())
31  print(elefante.animalToString())
32  print(gato.animalToString())
33  print(perro.animalToString())

```

La ejecución del programa es la siguiente:

```

Cebra 15000.0 150.0
Elefante 300000.0 2600.0
Gato 120.0 3.8
Perro 500.0 25.4

```

Se puede apreciar, que todos los animales creados son objetos de la clase Animal, pero todas han sido creadas de distinta forma.

Métodos estáticos

Los métodos estáticos pertenecen a la clase, pero no precisan hacer uso de la clase en sí ni de la instancia. Por lo tanto, no tienen parámetros principal, aunque acepta cualquier parámetro adicional como cualquier otro método o función. Se utilizan principalmente para unir código relacionado con una clase en particular, que podría estar definido de forma independiente a nivel de módulo, pero que se pretende tener ligado a la clase para mejorar la legibilidad o porque fuera de la clase no tiene sentido.

Hay múltiples ejemplos claros de métodos estáticos: documentación, fórmulas usadas en otros métodos de la clase o instancia, constantes, valores por defecto, etc.

Para definir un método estático se hace uso del decorador `@staticmethod` justo encima del método que se pretende definir. Suele ser muy utilizado para definir constantes de la clase.

Ejemplo, en la clase *Animal* definida en el ejemplo anterior se puede añadir los siguientes métodos que permiten calcular el peso de un animal a partir de su masa usando la constante de gravedad.

```
22
23     def peso(self):
24         return self.masa * Animal.gravedad()
25
26     @staticmethod
27     def gravedad():
28         return 9.8
29
```

```
print("El gato pesa: ", gato.peso())
```

```
Perro 500.0 25.4
El gato pesa: 37.24
```

Herencia

En nuestro ejemplo, acabas de comprar un coche con los atributos que buscabas. A pesar de ser únicos, hay autos con **exactamente los mismos atributos o formas modificadas**. Digamos que compraste el modelo Fit de Honda. Este modelo tiene otra versión llamada WR-V (o "Honda Fit Cross Style"), que tiene muchos de los atributos de la versión clásica, pero con algunas diferencias muy grandes para transitar por caminos de tierra: el motor es híbrido (acepta alcohol y gasolina), tiene un sistema de suspensión diferente, y supongamos que además tenga un sistema de tracción diferente (tracción a las cuatro ruedas, por ejemplo). Vemos entonces que no solo cambian algunos atributos sino también algunos mecanismos (o métodos, traducándose a POO), pero esta versión "cross" sigue siendo el modelo Honda Fit, es decir, es *un tipo* del modelo.

Cuando decimos que **una clase A es un tipo de clase B**, decimos que clase A *hereda* las características de la clase B y que la clase B es *madre* de la clase A, estableciendo entonces una relación de herencia entre ellas. En el caso del coche, decimos que un Honda Fit "Cross" es *un tipo de* Honda Fit, y lo que cambia son algunos atributos (tapabarros reforzado, altura de la suspensión, etc.), y uno de los métodos de la clase (acelerar, ya que ahora hay tracción en las cuatro ruedas), pero todo lo demás sigue igual, y el nuevo modelo recibe los mismos atributos y métodos que el modelo clásico.

Una de las herramientas más potentes de la programación orientada a objetos es la herencia. La herencia permite que, partiendo de una clase cualquiera, se pueda crear una nueva heredando sus métodos y atributos. El propósito es crear una clase nueva más específica que la clase original.

A la clase original de se le suele llamar "**clase base**" o "**clase padre**" y a la nueva clase se le llama "**clase hijo**" o "**heredera**". En Python las clases heredan por defecto de la clase *object*. Si se quiere hacer una herencia de clase distinta de *object*, si que hay que incluirlas en la definición de la misma.

A continuación, se muestra un ejemplo con tres clases. La clase padre *MiStr* pretende guardar una cadena de caracteres y tendrá una función que define la longitud de la cadena y una representación informacional con la función *info*. Las dos clases hijas extienden a la primera y aplican una función distinta para obtener la información, una lo hace utilizando las mayúsculas y la otra solo letras en minúscula:

```
1 class MiStr:
2     def __init__(self, cadena) -> None:
3         self.cadena = cadena
4
5     def longitud(self):
6         return self.cadena
7
8     def info(self):
9         return self.cadena
10
11 class MiStrMayusculas(MiStr):
12
13     def info(self):
14         return self.cadena.upper()
15
16 class MiStrMinusculas(MiStr):
17
18     def info(self):
19         return self.cadena.lower()
20
21 def main():
22
23     m_str = MiStr('Hola Mundo')
24     print('longitud del string', m_str.longitud())
25     print('contenido del string', m_str.info())
26
27     m_str_mayus = MiStrMayusculas('Hola Mundo')
28     print('longitud del string', m_str_mayus.longitud())
29     print('contenido del string', m_str_mayus.info())
30
31     m_str_minus = MiStrMinusculas('Hola Mundo')
32     print('longitud del string', m_str_minus.longitud())
33     print('contenido del string', m_str_minus.info())
34
35 if __name__ == '__main__':
36     main()
37
```

Al ejecutarlo el resultado es el siguiente:

```
longitud del string Hola Mundo
contenido del string Hola Mundo
longitud del string Hola Mundo
contenido del string HOLA MUNDO
longitud del string Hola Mundo
contenido del string hola mundo
```

Nótese que la función `longitud` está definida solamente en la clase padre, pero puede accederse desde cualquiera de las clases hijas. Esto es así porque cuando Python no encuentra un atributo o un método en la instancia en la que se está intentando acceder a él, va recorriendo la jerarquía de las clases padre intentando encontrarlo.

Si no lo consigue en ninguna, devuelve un error del tipo *AttributeError*. Es por esta misma razón, cuando se define el método *info* en cada una de las subclases, Python puede encontrar que ya hay un método definido en las instancias y, por tanto, ejecutarlo en vez de ejecutar de la clase padre.

A este concepto se le denomina **sobreescritura de métodos** y es de mucha utilidad cuando se quiere extender una clase con funcionalidades nuevas sin cambiar directamente la clase padre.

Esta sobreescritura también puede ser de atributos y funciona igual que la de métodos, cuando se define un atributo en una clase hija, Python lo devolverá al ser preguntado en vez de buscar el atributo en cualquiera de las clases padre.

La jerarquía de clases se puede comprobar haciendo uso de `__mro__` como se puede apreciar a continuación:

```
37 print(MiStrMayusculas.__mro__)
38 print(MiStrMinusculas.__mro__)
39 print(MiStr.__mro__)
```

```
(<class '__main__.MiStrMayusculas'>, <class '__main__.MiStr'>, <class 'object'>)
(<class '__main__.MiStrMinusculas'>, <class '__main__.MiStr'>, <class 'object'>)
(<class '__main__.MiStr'>, <class 'object'>)
```

Aquí se puede apreciar que la clase *MiStr* está definida en `__main__` (dado que se está ejecutando bajo la función *main*). Pero su clase padre es *object*. Sin embargo, en las clases hijas vemos que su padre es la clase *MiStr*.

Herencia y el uso de *super*

Cuando se hace una herencia, realmente se intenta hacer una versión de la clase padre más específica y, por tanto, se pretende aprovechar al máximo lo ya definido en ella. Con el fin de poder reutilizar el código de la clase padre, Python define un mecanismo para llamar a las funciones de esa clase y obtener los resultados desde la instancia hijo. Esto se hace mediante la función *super*.

La sintaxis para usar la instrucción de *super* es la siguiente:

`super(clase_hija, self).__init__(nombre, parámetros_clase_padre)`

aunque también podemos usar

`super().__init__(nombre, parámetros_clase_padre)`

Veamos un ejemplo de su uso:

Se quiere diseñar una clase *Persona*, la cual:

- Contiene el nombre y lo transforma a su forma de título en la inicialización de la instancia.
- Tiene dos métodos: uno que devuelve la descripción de la persona *info* y otro que devuelve la velocidad a la que puede andar *velocidad*

La clase *Atleta* es una clase que hereda de *Persona* y define a los atletas, la cual:

- Especifica mejor la información de los atletas
- Aumenta la velocidad que la persona tiene para andar basándose en un parámetro especial que tienen los atletas y que define su estado de forma.

La clase *Pintor*, que hereda de *Persona* y define a un artista, la cual:

- Especifica mejor la información de los artistas
- Tiene un nuevo método *pintar* para poder pintar utilizando caracteres ASCII.

El código que implementa las clases solicitadas es el que se muestra a continuación:

```

1 class Persona:
2     def __init__(self, nombre) -> None:
3         self.nombre = nombre.title()
4
5     def info(self):
6         return f'Mi nombre es: {self.nombre}'
7
8     def velocidad(self):
9         """Velocidad media en kilómetros por minutos"""
10        return 8
11
12 class Atleta(Persona):
13     def __init__(self, nombre, forma_fisica: float) -> None:
14         super(Atleta, self).__init__(nombre)
15         self.forma_fisica = forma_fisica
16
17     def info(self):
18         p_info = super(Atleta, self).info()
19         return f'{p_info} y soy atleta profesional'
20
21     def velocidad(self):
22         p_vel = super(Atleta, self).velocidad()
23         return p_vel * (1 + self.forma_fisica / 10)
24
25 class Pintor(Persona):
26     def info(self):
27         p_info = super(Pintor, self).info()
28         return f'{p_info} y soy pintor'
29
30     def pintar(self):
31         lineas = []
32         for vals in ([9556, 9552, 9552, 9559], [9553, 9630, 9626, 9553], [9562, 9552, 9552, 9565]):
33             lineas.append(''.join(map(chr, vals)))
34         return '\n'.join(lineas)

```

A destacar las clases *Atleta* y *Pintor* que en su definición se pasa como parámetro la clase *Persona*, con ello decimos que ambas clases descienden de la clase *Persona* y por tanto heredan todos los atributos y métodos de ésta.

En ambas clases hijas se sobrescribe el método `__init__` del padre, por tanto, para crear una instancia de los nuevos objetos se debe hacer uso de la instrucción *super*. Recuerden que se deben pasar todos los parámetros de la función `__init__` de la clase padre.

Si en la definición de la clase se quiere acceder a los atributos o métodos heredados de la clase padre, la forma de hacerlo es la siguiente:

`super(clase_hija, self).atributo` ó `super().atributo`
`super(clase_hija, self).método()` ó `super().metodo()`

Una ejecución del desarrollo de estas clases es el siguiente:

```
36 def main():
37
38     p = Persona('Juan Fernández')
39     print(p.info())
40     print(p.velocidad())
41
42     # forma_fisica = 12
43     a = Atleta('Ana María Chacón', forma_fisica = 12)
44     print(a.info())
45     print(a.velocidad())
46
47     pintor = Pintor('Diego Velázquez')
48     print(pintor.info())
49     print(pintor.velocidad())
50     print(pintor.pintar())
51
52 if __name__ == '__main__':
53     main()
```

Cuyo resultado es el que se muestra a continuación:

```
Mi nombre es: Juan Fernández
8
Mi nombre es: Ana María Chacón y soy atleta profesional
17.6
Mi nombre es: Diego Velázquez y soy pintor
8
```



Herencia múltiple

En Python se puede implementar la herencia múltiple. Esto quiere decir, que una clase puede tener más de un padre. A priori, esta característica puede resultar extraña y se puede convertir en un problema si se abusa de ella. Veamos cómo se puede implementar.

Como se explicó en el apartado anterior, para que una clase herede de otra solo hay que añadirla en la sentencia que crea la clase. Para que herede de más de una clase, hay que añadirlas en orden y separadas por comas.

Siguiendo el ejemplo anterior, se puede definir una clase de una persona que es tanto atleta como pintor, llamemos a esa clase *AtletaPintor*.


```

36 class AtletaPintor(Atleta, Pintor):
37     pass

```

Veamos ahora en ejecución una instancia de esta clase:

```

55 atletapintor = AtletaPintor("Juan González", forma_fisica=8)
56 print("\n")
57 print(atletapintor.info())
58 print("mi velocidad es de", atletapintor.velocidad())
59 print(atletapintor.pintar())

```

Cuyo resultado da:

```

Mi nombre es: Juan González y soy pintor y soy atleta profesional
mi velocidad es de 14.4

```



Para crear una clase que hereda tanto de *Atleta* como de *Pintor*, solo es necesario añadir las clases en la definición de las clases y automáticamente la nueva clase heredará todos los métodos y atributos de las clases superiores. Cabe destacar que la nueva clase *AtletaPintor* no hereda explícitamente de *Persona*, pero como sus clases superiores lo hacen, implícitamente también lo hace ella.

El orden que se establece en las herencias múltiples es algo a tener en cuenta, dado que, si dos clases implementan el mismo método, Python solo ejecutará la primera que encuentre ejecutando su MRO. En el ejemplo de *AtletaPintor* el orden que se establece es: primero la clase *Atleta* y, después, la de *Pintor*.

```

61 print('\n')
62 print(AtletaPintor.__mro__)

```

Cuyo resultado es:

```

(<class '__main__.AtletaPintor'>, <class '__main__.Atleta'>, <class '__main__.Pintor'>, <class '__main__.Persona'>, <class 'object'>)

```

Con esta definición de la clase *AtletaPintor*, cuando Python intenta ejecutar el método *velocidad* en la instancia de *AtletaPintor*, se busca en *Atleta*, donde encuentra la definición del método y la ejecuta devolviendo el resultado. Si no estuviera en *Atleta*, se seguiría buscando en *Pintor*, *Persona* y finalmente en *Object* y se elevaría una excepción si no está implementado.

Obviamente si se cambia el orden de las clases en su definición los resultados serían diferentes.

```

36 class AtletaPintor(Pintor, Atleta):
37     pass
38

```

Si ahora se define la clase como en la imagen anterior, el resultado de la ejecución del código de ejemplo anterior sería:

```

Mi nombre es: Juan González y soy atleta profesional y soy pintor
mi velocidad es de 14.4

(<class '__main__.AtletaPintor'>, <class '__main__.Pintor'>, <class '__main__.Atleta'>, <class '__main__.Persona'>, <class 'object'>)

```

Con esta definición de la clase *AtletaPintor*, cuando Python intenta ejecutar el método *velocidad* en la instancia de *AtletaPintor*, se busca en *Pintor*, donde no halla la definición del método buscado. Ahora se prosigue la búsqueda en *Atleta*, donde se encuentra y se ejecuta devolviendo el resultado. Si no estuviera se sigue buscando en *Persona* y finalmente en *Object* y se elevaría una excepción si no está implementado.

Gracias a que Python basa su ejecución en el orden que aparecen las clases superiores en el MRO es muy fácil de resolver el problema de las herencias múltiples que puedan tener métodos con el mismo nombre.

Veamos otro ejemplo:

Ahora se quiere definir una clase de nombre *PintorRapido*, que, si va a implementar los métodos de *velocidad* e *info*, y se utilizará para definir las clases *PintorRapidoAtleta* como *AtletaPintorRapido*. El código es el siguiente:

```

36 class PintorRapido(Pintor):
37
38     def info(self):
39         p_info = super(Pintor, self).info()
40         return f'{p_info} y soy pintor rápido'
41
42     def velocidad(self):
43         vel = super(PintorRapido, self).velocidad()
44         return vel + 2
45
46 class PintorRapidoAtleta(PintorRapido, Atleta):
47     pass
48
49 class AtletaPintorRapido(Atleta, PintorRapido):
50     pass

```

Y el siguiente código para su ejecución:

```

67     pra = PintorRapidoAtleta("Juana", forma_fisica = 45)
68
69     print("\n")
70     print(pra.info())
71     print("mi velocidad es de", pra.velocidad())
72     print(pra.pintar())
73
74     apr = AtletaPintorRapido("María", forma_fisica = 45)
75
76     print("\n")
77     print(apr.info())
78     print("mi velocidad es de", apr.velocidad())
79     print(apr.pintar())

```

Cuyo resultado es:

```

Mi nombre es: Juana y soy atleta profesional y soy pintor rápido
mi velocidad es de 46.0

```



```

Mi nombre es: María y soy pintor rápido y soy atleta profesional
mi velocidad es de 55.0

```



Y sus correspondientes MRO

```

(<class '__main__.PintorRapidoAtleta'>, <class '__main__.PintorRapido'>, <class '__main__.Pintor'>, <class '__main__.Atleta'>, <class '__main__.Persona'>, <class 'object'>)

```

```

(<class '__main__.AtletaPintorRapido'>, <class '__main__.Atleta'>, <class '__main__.PintorRapido'>, <class '__main__.Pintor'>, <class '__main__.Persona'>, <class 'object'>)

```

Como se puede ver en el ejemplo, cuando se pregunta por el método *info* en la instancia para la clase *PintorRapidoAtleta*, las llamadas se producen como indica el mro, pero el resultado es contrario a lo que se puede pensar, dado que la información la está dando en

el orden *Persona*, *Atleta* y *PintorRapido* en vez de como marca el MRO correspondiente (*PintorRapido*, *Atleta*, *Persona*). Esto se debe a que cada método *info* de la clase hace una llamada a *super* en la primera sentencia y va concatenando las cadenas de caracteres resultantes en el orden inverso al que se han producido las llamadas.

Lo mismo pasa con las llamadas a velocidad, notar que en las dos instancias *pra* y *apr* dan resultados distintos, esto es debido al orden de las llamadas que se producen por el orden en que se han creado la multiplicidad de la herencia de clases en ellas.

Para la instancia *pra* el resultado se obtiene así:

pra es instancia de la clase *PintorRapidoAtleta* que deriva de *PintorRapido* y *Atleta* en ese orden. Cuando se ejecuta el método *velocidad* de la instancia, primero se invoca el método *velocidad* de la clase *Atleta*, código que repetimos aquí.

```
def velocidad(self):  
    p_vel = super(Atleta, self).velocidad()  
    return p_vel * (1 + self.forma_fisica / 10)
```

Como este método sobrescribe el método *velocidad* de la clase *Persona* se invoca su método mediante *super()* haciendo que *p_vel*=8 y haciendo cuentas lo que retorna es $8 \cdot (1 + 45/10) = 44$, cuando regresa a la clase *PintorRapido* recoge el 44 y le suma 2 para dar un total de 46, que es lo que se muestra por pantalla.

Para la instancia *apr* el resultado se obtiene así:

apr es instancia de la clase *AtletaPintorRapido* que deriva de *Atleta* y *PintorRapido* en ese orden. Cuando se ejecuta el método *velocidad* de la instancia, primero se invoca el método *velocidad* de la clase *PintorRapido*, código que repetimos aquí.

```
def velocidad(self):  
    vel = super(PintorRapido, self).velocidad()  
    return vel + 2
```

La clase *PintorRapido* hereda de *Pintor* que no tiene método *velocidad*, pero que hereda de *Persona* que si lo tiene por tanto *vel*=8 que sumándole 2 retornará el valor de 10.

Este valor de 10 es recogido en la clase *Atleta* (la otra clase de la que hereda y que si implementa el método *velocidad*, reproducido antes), en este caso *p_vel*=10 y retornará $10 \cdot (1 + 45/10) = 55$ que es lo que muestra por pantalla.

Este proceso se puede complicar bastante, si intervinieran más clases que tengan definidos atributos o métodos comunes, por eso se recomienda hacer un uso moderado de la misma.

Veamos otro ejemplo:

```
1  class Animals:
2      def __init__(self, animalsName):
3          print(animalsName, 'is an animal.');
```

```
4
5
6  class Mammal(Animals):
7      def __init__(self, Name):
8          print(Name, 'is a mammal.')
9          super().__init__(Name)
```

```
10
11
12 class donotFly(Mammal):
13     def __init__(self, name):
14         print(name, "cannot fly.")
15         super().__init__(name)
```

```
16
17
18 class donotSwim(Mammal):
19     def __init__(self, name):
20         print(name, "cannot swim.")
21         super().__init__(name)
```

```
22
23
24 class Cat(donotSwim, donotFly):
25     def __init__(self):
26         print('A cat.');
```

```
27         super().__init__('Cat')
```

```
28
29 cat = Cat()
30 print('')
31 bat = donotSwim('Bat')
```

Se crea una clase secundaria Cat, heredada por las clases biparentales donotswim y donotfly. Entonces, la clase Mammals los hereda ellos mismos. Además, la clase Mammals hereda las propiedades de la superclase Animals. Por lo tanto, en este caso, usamos la función super() para acceder fácilmente a los métodos de la superclase. Los print en el ejemplo es para ver la secuencia de llamadas, el resultado del código anterior es:

```

Ign: Informatica@macb
A cat.
Cat cannot swim.
Cat cannot fly.
Cat is a mammal.
Cat is an animal.

Bat cannot swim.
Bat is a mammal.
Bat is an animal.

```

Polimorfismo (duck typing)

En la programación orientada a objetos existe un concepto denominado **polimorfismo**. Se basa en que objetos de diferentes tipos pueden tener características similares y, por tanto, compartir nombres de métodos o atributos.

El caso de uso más común para el polimorfismo es poder compartir código entre objetos de tipos distintos, que al comportarse de forma similar con los mismos métodos y usando las mismas funciones son intercambiables entre sí.

Un ejemplo simple puede ser la definición de diferentes clases que tengan el mismo método, pero que devuelvan resultados diferentes, y utilizar una función única que usará el polimorfismo y llamará a ese método que comparte el mismo identificador, pero en cada una de las instancias.

Veamos el siguiente ejemplo:

```

1  class FormateadorMayusculas:
2      def formatea(self, cadena):
3          return cadena.upper()
4
5  class FormateadorMinusculas:
6      def formatea(self, cadena):
7          return cadena.lower()
8
9  def formatear(objeto, cadena):
10     return objeto.formatea(cadena)
11
12  f_may = FormateadorMayusculas()
13  f_min = FormateadorMinusculas()
14
15  print('\n'+ formatear(f_may, 'Esto Tiene Letras De Los Dos Tipos'))
16  print(['\n'+ formatear(f_min, 'Esto Tiene Letras De Los Dos Tipos')])
17

```

Como puede verse, la función formatear simplemente ejecuta el mismo método para el objeto que se le pasa como primer argumento, sin reparar el tipo de objeto que es.

Otro ejemplo se presenta cuando se hace uso de la herencia y varias clases comparten los atributos de la misma clase padre (o clase base), por lo que se define o redefine un método que tiene la clase base, como se puede apreciar en el siguiente ejemplo:

```
1  class NumeroDecimal:
2      def __init__(self, numero_original) -> None:
3          self._num = numero_original
4
5      def numero(self):
6          return self._num
7
8  class NumeroBinario(NumeroDecimal):
9
10     def numero(self):
11         return bin(self._num)
12
13  class NumeroHexadecimal(NumeroDecimal):
14
15     def numero(self):
16         return hex(self._num)
17
18  # principal
19
20  b = NumeroBinario(78)
21  h = NumeroHexadecimal(78)
22
23  print('\n' + b.numero())
24  print('\n' + h.numero())
```

Que presenta la siguiente salida:

```
0b1001110
0x4e
```

El apelativo de duck typing viene de una expresión que indica que “si un ave camina como un pato, nada como un pato y suena como un pato, a esa ave la llamaré pato”. Se puede aplicar a este concepto de polimorfismo que implementa Python porque, sin importar el tipo del objeto al que se le aplique, si un atributo define el método y responde a él, se puede utilizar el objeto.

Clases abstractas en Python

Una clase abstracta puede considerarse como un modelo para otras clases. Le permite crear un conjunto de métodos que deben crearse dentro de las clases secundarias creadas a partir de la clase abstracta. Una clase que contiene uno o más métodos abstractos se llama clase abstracta. Un método abstracto es un método que tiene una declaración pero no tiene una implementación. Mientras diseñamos grandes unidades funcionales, usamos una clase abstracta. Cuando queremos proporcionar una interfaz común para diferentes implementaciones de un componente, usamos una clase abstracta.

Por qué utilizar clases base abstractas:

Al definir una clase base abstracta, puede definir una interfaz de programa de aplicación(API) común para un conjunto de subclases. Esta capacidad es especialmente útil en situaciones en las que un tercero proporcionará implementaciones, como con complementos, pero también puede ayudarlo cuando trabaja en un equipo grande o con una base de código grande donde es difícil mantener todas las clases en su mente. o no es posible.

Cómo funcionan las clases de Abstract Base:

De forma predeterminada, Python no proporciona clases abstractas. Python viene con un módulo que proporciona la base para definir las clases base abstractas (ABC) y el nombre del módulo es ABC. **ABC** funciona decorando métodos de la clase base como abstractos y luego registrando clases concretas como implementaciones de la base abstracta. Un método se vuelve abstracto cuando se decora con la palabra clave @abstractmethod.

Por lo tanto, es muy útil usar una clase abstracta para definir una interfaz común para diferentes implementaciones.

Una clase abstracta tiene algunas características, como sigue:

- Una clase abstracta no contiene todas las implementaciones de métodos necesarias para funcionar completamente, lo que significa que contiene uno o más métodos abstractos. Un método abstracto es un método que solo tiene una declaración, pero no tiene una implementación detallada.
- No se puede crear una instancia de una clase abstracta. Simplemente proporciona una interfaz para subclases para evitar la duplicación de código. No tiene sentido crear una instancia de una clase abstracta.
- Una subclase derivada debe implementar los métodos abstractos para crear una clase concreta que se ajuste a la interfaz definida por la clase abstracta. Por lo tanto, no se puede crear una instancia a menos que se anulen todos sus métodos abstractos.

Por ejemplo:

Python viene con un módulo llamado abc que proporciona cosas útiles para la clase abstracta.

Podemos definir una clase como una clase abstracta por `abc.ABC` y definir un método como un método abstracto por `@abstractmethod`. `ABC` es la abreviatura de clase base abstracta.

```
1  from abc import ABC, abstractmethod
2
3  class Polygon(ABC):
4
5      @abstractmethod
6      def noofsides(self):
7          pass
8
9  class Triangle(Polygon):
10
11      # overriding abstract method
12      def noofsides(self):
13          print("I have 3 sides")
14
15  class Pentagon(Polygon):
16
17      # overriding abstract method
18      def noofsides(self):
19          print("I have 5 sides")
20
21  class Hexagon(Polygon):
22
23      # overriding abstract method
24      def noofsides(self):
25          print("I have 6 sides")
26
27  class Quadrilateral(Polygon):
28
29      # overriding abstract method
30      def noofsides(self):
31          print("I have 4 sides")
32
```

El programa que crea las instancias:

```

33 # principal
34
35 R = Triangle()
36 R.noofsides()
37
38 K = Quadrilateral()
39 K.noofsides()
40
41 R = Pentagon()
42 R.noofsides()
43
44 K = Hexagon()
45 K.noofsides()
46
47 print("-----")
48
49 P = Polygon()

```

Con el siguiente resultado:

```

I have 3 sides
I have 4 sides
I have 5 sides
I have 6 sides
-----
Traceback (most recent call last):
  File "/Users/rgh.informatica/Documents/ACADEMICO/CURSO_21-22/DAM/00-FICHEROS-PYTHON/EjemploTema/ClaseAbstracta.py", line 49, in <module>
    P = Polygon()
TypeError: Can't instantiate abstract class Polygon with abstract method noofsides

```

Para poder crear una clase abstracta en Python se debe importar *ABC* de la librería *abc*, además se necesita importar también *abstractmethod*.

La clase *Polygon* del ejemplo anterior, es una clase abstracta, pues hereda de la clase *ABC*, para definir un método como abstracto se debe utilizar la propiedad *abstractmethod* antes del método a definir.

En una clase abstracta, **sólo se definen las interfaces de los métodos**, por eso el uso de la instrucción *pass* como única instrucción del método. Es en las clases que luego hereden de la clase abstracta, **las que están obligadas a desarrollar todos los métodos que sean declarados como abstractos**.

En la ejecución del programa, el error viene dado porque **una clase abstracta NO permite que se realicen instancias de ellas**, de hecho, no tiene constructor.

Métodos mágicos

Los métodos mágicos son métodos creados para poder definir protocolos comunes a todas las clases y tipos de datos, tanto del núcleo de Python como creadas individualmente.

Las operaciones de comparación, los operadores matemáticos están implementados por métodos especiales que se implementan en las clases para poder realizarlo.

Un uso común de estos es la sobrecarga de operadores. Esto significa definir operadores para clases personalizadas que permiten que operadores tales como `+` o `*` sean utilizados en ellas. Un ejemplo de un método mágico es `__add__()` para `+`.

Algunos de los métodos que se pueden implementar en las clases que se diseñan son las siguientes:

Métodos asociados a los operadores relacionales:

- `__eq__(self, objeto)`: que sirve para comparar si el objeto instanciado en `self` es igual al instanciado en `objeto`.
- `__ne__(self, objeto)`: que sirve para comparar si el objeto instanciado en `self` es distinto al instanciado en `objeto`.
- `__gt__(self, objeto)`: que sirve para comparar si el objeto instanciado en `self` es mayor al instanciado en `objeto`.
- `__ge__(self, objeto)`: que sirve para comparar si el objeto instanciado en `self` es mayor o igual al instanciado en `objeto`.
- `__lt__(self, objeto)`: que sirve para comparar si el objeto instanciado en `self` es menor al instanciado en `objeto`.
- `__le__(self, objeto)`: que sirve para comparar si el objeto instanciado en `self` es menor o igual al instanciado en `objeto`.

Métodos asociados a los operadores aritméticos:

- `__add__(self, objeto)`: que sirve para realizar la suma de los dos objetos.
- `__sub__(self, objeto)`: que sirve para realizar la resta (`self - objeto`) de los dos objetos.
- `__mul__(self, objeto)`: que sirve para realizar la multiplicación de los dos objetos.
- `__truediv__(self, objeto)`: que sirve para realizar la división (`self / objeto`) de los dos objetos.
- `__floordiv__(self, objeto)`: que sirve para realizar la división entera (`self // objeto`) de los dos objetos.
- `__mod__(self, objeto)`: que sirve para realizar la operación módulo, resto de la división entera `self % objeto` de los dos objetos.
- `__pow__(self, objeto)`: que sirve para realizar la potencia (`selfobjeto`) de los dos objetos.

Veamos un ejemplo del uso de estos métodos mágicos en una clase diseñada por nosotros:

Crear una clase jugador con los atributos nombre y puntos. Crear un método que permita asignarle puntos a un jugador. También se quiere un método que permita sumar los puntos de dos jugadores. Diseñe el método `__str__()` que permita mostrar los datos de un jugador de una forma amigable

El código para definir la clase y los métodos es el siguiente:

```
1 ~ class Jugador:
2 ~     def __init__(self, nombre, puntuacion = 0) -> None:
3 ~         self.nombre = nombre
4 ~         self.puntos = puntuacion
5 ~
6 ~     def asignarPuntuacion(self, puntuacion):
7 ~         self.puntos = puntuacion
8 ~
9 ~     def __str__(self) -> str:
10 ~         return "\nNombre jugador: " + self.nombre + " tiene " + str(self.puntos) + " puntos"
11 ~
12 ~     def __add__(self, objeto):
13 ~         return self.puntos + objeto.puntos
14
```

Y con el siguiente código para probarlo:

```
17 jug01 = Jugador("Alberto")
18 jug02 = Jugador("Pedro")
19 jug01.asignarPuntuacion(13)
20 jug02.asignarPuntuacion(14)
21 jug03 = Jugador("Maria")
22 jug03.asignarPuntuacion( jug01 + jug02 )
23 print(jug01.__str__())
24 print(jug02.__str__())
25 print(jug03.__str__())
26 print("\n")
```

El resultado obtenido es el siguiente:

```
Nombre jugador: Alberto tiene 13 puntos

Nombre jugador: Pedro tiene 14 puntos

Nombre jugador: Maria tiene 27 puntos
```

Lo más destacado del código de prueba es la suma realizado entre las dos objetos instanciados de la clase jugador. Esto es posible porque se ha definido el método mágico `__add__(self, objeto)`. Como se puede verificar el resultado se ha comportado como si fuera una suma normal.

De igual forma, para verificar si dos jugadores tienen la misma puntuación se puede implementar el método `__eq__(self, objeto)` tal y como se muestra en la figura:

```
14  
15     def __eq__(self, objeto):  
16         return self.puntos == objeto.puntos  
17
```

La forma de usar ese método sería la siguiente:

```
29  
30     print(jug03 == jug02)  
31
```

Que produce la siguiente respuesta:

```
Nombre jugador: Alberto tiene 13 puntos  
Nombre jugador: Pedro tiene 14 puntos  
Nombre jugador: Maria tiene 27 puntos  
False
```